
LAB Task

Classes, Objects, Data Members & Member Functions

Name: Hania

Roll No: SP26-BAI-024

1. Environment Setup and Compilation Procedure

All programs in this lab were written in a plain text editor and compiled and executed from the Windows Command Prompt. The IDE Run button was not used at any point.

Step 1 - Open Command Prompt and move into the lab folder

```
cd C:\CSC241\CIIT-SP26-BAI-024-LHR
mkdir Lab02
cd Lab02
```

Step 2 - Verify that the JDK and Git are installed

```
java -version
javac -version
git --version
```

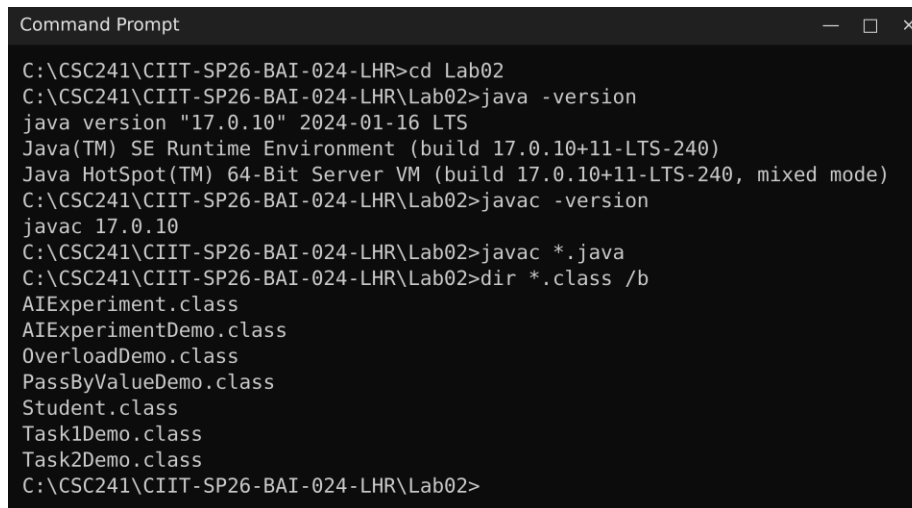
Step 3 - Compile every source file in the folder

```
javac *.java
```

The wildcard *.java compiles all files together. This matters because Task1Demo, Task2Demo and PassByValueDemo depend on the Student class, and AIExperimentDemo depends on AIExperiment. Compiling a single demo file on its own would produce a "cannot find symbol" error. A successful compilation prints nothing and simply returns to the prompt, producing one .class file per class.

Step 4 - Run each program by class name (no .java or .class extension)

```
java Task1Demo
java Task2Demo
java OverloadDemo
java PassByValueDemo
java AIExperimentDemo
```



```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR>cd Lab02
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>java -version
java version "17.0.10" 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 17.0.10+11-LTS-240)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.10+11-LTS-240, mixed mode)
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac -version
javac 17.0.10
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac *.java
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>dir *.class /b
AIExperiment.class
AIExperimentDemo.class
OverloadDemo.class
PassByValueDemo.class
Student.class
Task1Demo.class
Task2Demo.class
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

Common errors and their meaning

- 'javac' is not recognized - the JDK bin folder is not on the Windows PATH.
- error: cannot find symbol, class Student - Student.java was not compiled or is in a different folder.
- Could not find or load main class Task1Demo.class - the program was run with the .class extension; use java Task1Demo instead.

2. Task 1 - Build a Class and Prove Independent Object State

Objective: create one Student class with three instance variables, instantiate three separate objects from it, and demonstrate that modifying one object leaves the other two untouched.

2.1 Source Code

Student.java

```
class Student {
    String studentId;
    String name;
    int completedCredits;

    void addCredits(int credits) {
        completedCredits = completedCredits + credits;
    }

    int remainingCredits(int degreeCredits) {
        int remaining = degreeCredits - completedCredits;
        return remaining;
    }

    String summary() {
        return studentId + " | " + name + " | completedCredits = " + completedCredits;
    }
}
```

Note: the three member functions shown above are added in Task 2. The class used in Task 1 consists of the three instance variables only.

Task1Demo.java

```
class Task1Demo {

    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student();
        Student s3 = new Student();

        s1.studentId = "SP26-BAI-024";
        s1.name = "Hania";
        s1.completedCredits = 18;

        s2.studentId = "SP26-BAI-025";
        s2.name = "Ali Ishtiaq";
        s2.completedCredits = 21;

        s3.studentId = "SP26-BAI-026";
        s3.name = "Abdul Rehman Azam";
        s3.completedCredits = 15;

        System.out.println("=== BEFORE CHANGE ===");
        System.out.println(s1.studentId + " " + s1.name + " credits = " + s1.completedCredits);
        System.out.println(s2.studentId + " " + s2.name + " credits = " + s2.completedCredits);
        System.out.println(s3.studentId + " " + s3.name + " credits = " + s3.completedCredits);

        s2.completedCredits = s2.completedCredits + 9;

        System.out.println();
        System.out.println("=== AFTER CHANGING ONLY s2 ===");
```

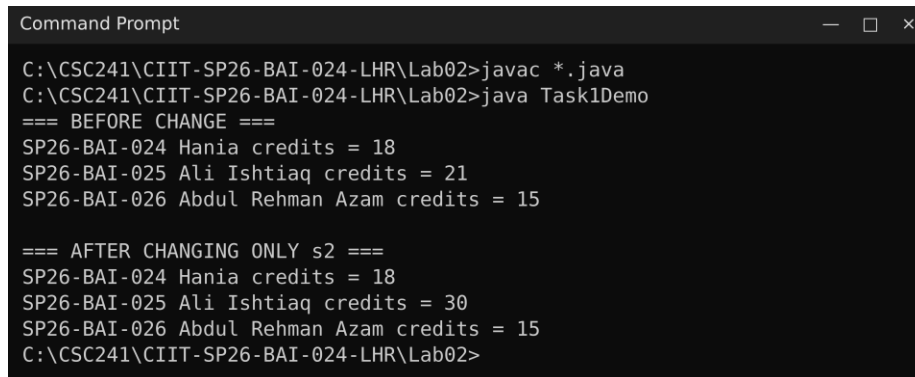
```

System.out.println(s1.studentId + " " + s1.name + " credits = " + s1.completedCredits);
System.out.println(s2.studentId + " " + s2.name + " credits = " + s2.completedCredits);
System.out.println(s3.studentId + " " + s3.name + " credits = " + s3.completedCredits);

// s1, s2 and s3 refer to three different objects, and each object keeps its own
// copy of completedCredits, so writing through s2 changes only that object.
}

```

2.2 Command Prompt Output



```

Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac *.java
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>java Task1Demo
=== BEFORE CHANGE ===
SP26-BAI-024 Hania credits = 18
SP26-BAI-025 Ali Ishtiaq credits = 21
SP26-BAI-026 Abdul Rehman Azam credits = 15

=== AFTER CHANGING ONLY s2 ===
SP26-BAI-024 Hania credits = 18
SP26-BAI-025 Ali Ishtiaq credits = 30
SP26-BAI-026 Abdul Rehman Azam credits = 15
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>

```

2.3 Concept Check

Q1. What is the difference between the Student class and the objects s1, s2 and s3 created from it?

Answer: Student is the blueprint that describes what fields every student has, but it stores no data itself. s1, s2 and s3 are three actual objects created in memory from that blueprint, each with its own copy of the fields.

Q2. Why does changing s2.completedCredits not change the completedCredits value stored in s1 or s3?

Answer: Every object owns a separate copy of completedCredits, and the expression s2.completedCredits writes only into the object that s2 refers to. The other two objects are at different memory locations and are never touched.

Q3. Which variables in Student represent persistent object state, and why are they instance variables rather than local variables?

Answer: studentId, name and completedCredits hold the persistent state. They are instance variables so that they live as long as the object does; a local variable would be destroyed the moment its method returned and could not carry state between method calls.

3. Task 2 - Parameters, Local Variables and Return Values

Objective: extend Student with three member functions - one void method that changes state, one method that returns a computed int using a local variable, and one method that returns a readable String summary - then call all three on two different objects.

3.1 Source Code

The three methods are part of Student.java shown in Task 1: addCredits(int), remainingCredits(int) and summary(). The driver class is listed below.

Task2Demo.java

```
class Task2Demo {

    public static void main(String[] args) {

        int degreeCredits = 133;

        Student first = new Student();
        first.studentId = "SP26-BAI-024";
        first.name = "Hania";
        first.completedCredits = 18;

        Student second = new Student();
        second.studentId = "SP26-BAI-025";
        second.name = "Ali Ishtiaq";
        second.completedCredits = 21;

        System.out.println("=== INITIAL STATE ===");
        System.out.println(first.summary());
        System.out.println(second.summary());

        first.addCredits(6);
        second.addCredits(3);

        System.out.println();
        System.out.println("=== AFTER addCredits ===");
        System.out.println(first.summary());
        System.out.println(second.summary());

        int firstRemaining = first.remainingCredits(degreeCredits);
        int secondRemaining = second.remainingCredits(degreeCredits);

        System.out.println();
        System.out.println("=== REMAINING CREDITS (degree = " + degreeCredits + ") ===");
        System.out.println(first.name + " still needs " + firstRemaining + " credits");
        System.out.println(second.name + " still needs " + secondRemaining + " credits");
    }
}
```

3.2 Command Prompt Output

```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac *.java
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>java Task2Demo
=== INITIAL STATE ===
SP26-BAI-024 | Hania | completedCredits = 18
SP26-BAI-025 | Ali Ishtiaq | completedCredits = 21

=== AFTER addCredits ===
SP26-BAI-024 | Hania | completedCredits = 24
SP26-BAI-025 | Ali Ishtiaq | completedCredits = 24

=== REMAINING CREDITS (degree = 133) ===
Hania still needs 109 credits
Ali Ishtiaq still needs 109 credits
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

3.3 Field, Parameter and Local Variable

- Field: completedCredits - belongs to the object and keeps its value after the method finishes.
- Parameter: credits and degreeCredits - receive a copy of the argument at the moment of the call.
- Local variable: remaining, firstRemaining, secondRemaining - created inside the method and destroyed when it returns.

3.4 Concept Check

Q1. In addCredits(int credits), what is the difference between the parameter credits and the object field completedCredits?

Answer: credits is a parameter: a temporary copy of the argument that exists only while the method runs. completedCredits is a field belonging to the receiver object, so the value written into it survives after the call ends.

Q2. Why is remaining inside remainingCredits(...) better as a local variable than as an instance field?

Answer: It is scratch work for a single calculation and is recomputed on every call. Making it a field would waste memory in every object and would allow a value left over from one call to leak into the next.

Q3. What information does the return type of remainingCredits(int degreeCredits) give to the caller?

Answer: The return type int promises the caller that the method hands back a whole number which can be stored in a variable, printed, or used in further arithmetic, unlike a void method which returns nothing.

4. Task 3 - Method Signatures and Overloading

Objective: write three valid overloads of enroll, predict and test valid and invalid calls, and deliberately attempt an overload that differs only by return type in order to capture the compiler error.

4.1 Source Code

OverloadDemo.java

```
class OverloadDemo {

    // Signature: enroll(String)
    void enroll(String courseCode) {
        System.out.println("Enrolled in course " + courseCode + " (default section)");
    }

    // Signature: enroll(String, int)
    void enroll(String courseCode, int section) {
        System.out.println("Enrolled in course " + courseCode + " section " + section);
    }

    // Signature: enroll(int)
    void enroll(int numericCourseCode) {
        System.out.println("Enrolled using numeric course code " + numericCourseCode);
    }

    // Invalid overload: same signature as enroll(String), differs only by return type.
    // int enroll(String courseCode) {
    //     return 1;
    // }

    void show(long value) {
        System.out.println("show(long) selected with " + value);
    }

    void show(double value) {
        System.out.println("show(double) selected with " + value);
    }

    public static void main(String[] args) {

        OverloadDemo demo = new OverloadDemo();

        demo.enroll("CSC241");
        demo.enroll("CSC241", 2);
        demo.enroll(241);

        // demo.enroll();
        // demo.enroll("241", "2");

        demo.show(10);
    }
}
```

4.2 Predicted Calls

- enroll("CSC241") selects enroll(String) - one String argument.
- enroll("CSC241", 2) selects enroll(String, int) - two arguments, String then int.
- enroll(241) selects enroll(int) - one int argument.

- `enroll()` does not compile - no zero-parameter overload exists.
- `enroll("241", "2")` does not compile - `String` cannot be converted to `int`.
- Stretch: `show(10)` selects `show(long)`, because `int` widens to `long` before it widens to `double` and the compiler chooses the most specific applicable method.

4.3 Command Prompt Output

```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac *.java
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>java OverloadDemo
Enrolled in course CSC241 (default section)
Enrolled in course CSC241 section 2
Enrolled using numeric course code 241
show(long) selected with 10
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

4.4 Invalid Overload - Compiler Evidence

An extra method with the same name and the same parameter list, differing only in return type, was added temporarily:

```
void enroll(String courseCode) { ... }
int  enroll(String courseCode) { return 1; }
```

```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac OverloadDemo.java
OverloadDemo.java:23: error: method enroll(String) is already defined in class OverloadDemo
    int enroll(String courseCode) {
        ^
1 error
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

The method signature consists of the method name plus the parameter list only. Because the return type is not part of the signature, both methods have the identical signature `enroll(String)`, and at a call site such as `enroll("CSC241")`; where the result is discarded the compiler would have no information with which to choose between them. The invalid method was commented out afterwards.

```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac OverloadDemo.java
OverloadDemo.java:44: error: no suitable method found for enroll(no arguments)
    demo.enroll();
    ^
    method OverloadDemo.enroll(String) is not applicable
      (actual and formal argument lists differ in length)
    method OverloadDemo.enroll(String,int) is not applicable
      (actual and formal argument lists differ in length)
    method OverloadDemo.enroll(int) is not applicable
      (actual and formal argument lists differ in length)
OverloadDemo.java:45: error: no suitable method found for enroll(String,String)
    demo.enroll("241", "2");
    ^
    method OverloadDemo.enroll(String,int) is not applicable
      (argument mismatch; String cannot be converted to int)
2 errors
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

4.5 Concept Check

Q1. What is the method signature of `enroll(String courseCode, int section)`?

Answer: `enroll(String, int)` - the method name together with the number, types and order of its parameters. Parameter names and the return type are not part of the signature.

Q2. Why are `enroll(String)` and `enroll(int)` valid overloads, while two methods with the same name and parameter types but different return types are not?

Answer: `enroll(String)` and `enroll(int)` differ in parameter type, so the compiler can decide which one is meant from the argument at the call site. Two methods differing only by return type have identical signatures, and a call that discards the result would give the compiler nothing to choose on.

Q3. For the call `enroll("CSC241", 2)`, which overload should the compiler select, and what information does it use?

Answer: It selects `enroll(String, int)`, using the number of arguments (two) and their compile-time types (String followed by int). Overload resolution happens at compile time, not at run time.

5. Task 4 - Proving Java Pass-by-Value

Objective: run three controlled experiments showing that a primitive value is copied, that a reference value is also copied but still points at the same object, and that reassigning the parameter cannot redirect the caller's variable.

5.1 Source Code

PassByValueDemo.java

```
class PassByValueDemo {

    void changeNumber(int x) {
        System.out.println(" Inside changeNumber, x received = " + x);
        x = 99;
        System.out.println(" Inside changeNumber, x after assignment = " + x);
    }

    void changeStudent(Student st) {
        System.out.println(" Inside changeStudent, credits received = " + st.completedCredits);
        st.completedCredits = 99;
        System.out.println(" Inside changeStudent, credits after mutation = " + st.completedCredits);
    }

    void replaceStudent(Student st) {
        System.out.println(" Inside replaceStudent, name received = " + st.name);
        st = new Student();
        st.name = "Temporary";
        System.out.println(" Inside replaceStudent, parameter now points to = " + st.name);
    }

    public static void main(String[] args) {

        PassByValueDemo demo = new PassByValueDemo();

        int credits = 18;
        System.out.println("EXPERIMENT A - primitive");
        System.out.println(" Before : credits = " + credits);
        demo.changeNumber(credits);
        System.out.println(" After  : credits = " + credits);

        Student student = new Student();
        student.studentId = "SP26-BAI-024";
        student.name = "Hania";
        student.completedCredits = 18;

        System.out.println();
        System.out.println("EXPERIMENT B - object mutation");
        System.out.println(" Before : " + student.summary());
        demo.changeStudent(student);
        System.out.println(" After  : " + student.summary());

        System.out.println();
        System.out.println("EXPERIMENT C - parameter reassignment");
        System.out.println(" Before : " + student.summary());
        demo.replaceStudent(student);
        System.out.println(" After  : " + student.summary());

        // In every call the argument value is copied into the parameter.
    }
}
```

5.2 Command Prompt Output

```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac *.java
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>java PassByValueDemo
EXPERIMENT A - primitive
  Before : credits = 18
  Inside changeNumber, x received = 18
  Inside changeNumber, x after assignment = 99
  After  : credits = 18

EXPERIMENT B - object mutation
  Before : SP26-BAI-024 | Hania | completedCredits = 18
  Inside changeStudent, credits received = 18
  Inside changeStudent, credits after mutation = 99
  After  : SP26-BAI-024 | Hania | completedCredits = 99

EXPERIMENT C - parameter reassignment
  Before : SP26-BAI-024 | Hania | completedCredits = 99
  Inside replaceStudent, name received = Hania
  Inside replaceStudent, parameter now points to = Temporary
  After  : SP26-BAI-024 | Hania | completedCredits = 99
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

5.3 Before / Inside / After Table

Experiment A (int)	Before: 18 Inside: 18 then 99 After: 18 (unchanged)
Experiment B (mutation)	Before: 18 Inside: 99 After: 99 (change is visible)
Experiment C (reassign)	Before: Hania / 99 Inside: Temporary After: Hania / 99 (unchanged)

5.4 Required Explanation

In every Java method call the argument value is copied into the parameter. In Experiment A the copied value is the number 18, so assigning 99 to the parameter overwrites only that copy and the caller's variable keeps its old value. In Experiment B the copied value is a reference that points at the same Student object on the heap, so writing `st.completedCredits = 99` writes into the very object the caller is also holding, and the change is visible after the call. In Experiment C the statement `st = new Student()` replaces the reference value stored in the parameter copy only; the caller's variable holds its own copy of the original reference and is never affected. This is why `changeStudent` can affect the object while `replaceStudent` cannot redirect the caller's variable. Java does not pass objects by reference - it passes reference values by value.

5.5 Concept Check

Q1. After `changeNumber(int x)` assigns `x = 99`, why does the caller's original int variable keep its old value?

Answer: Because the argument value is copied into the parameter, `x` is a completely separate variable, and assigning 99 overwrites only that copy while the caller's variable is untouched.

Q2. If a Student reference is passed to a method, why can changing `st.completedCredits` be visible to the caller?

Answer: The copied value is a reference pointing at the same object in the heap, so a field written through that copy is written into the object that the caller is also looking at.

Q3. Why does assigning `st = new Student()` inside a method not make the caller's variable refer to the new object?

Answer: The assignment only replaces the reference value stored inside the parameter, which is a local copy. The caller's variable stores its own copy of the old reference and still points at the original object.

6. Task 5 - Integrated BAI Challenge: AIExperiment

Objective: design a small class that tracks the progress of AI experiments, using only the concepts covered through Lecture 4 - instance variables, instance methods, an overloaded method, a local variable, and a pass-by-value demonstration.

6.1 Source Code

AIExperiment.java

```
class AIExperiment {

    String experimentName;
    int completedEpochs;
    int targetEpochs;

    void runEpochs(int epochs) {
        completedEpochs = completedEpochs + epochs;
    }

    void runEpochs(int epochs, int bonusEpochs) {
        int totalThisRun = epochs + bonusEpochs;
        completedEpochs = completedEpochs + totalThisRun;
    }

    int remainingEpochs() {
        int remaining = targetEpochs - completedEpochs;
        return remaining;
    }

    String status() {
        return experimentName + " : " + completedEpochs + "/" + targetEpochs
            + " epochs done, " + remainingEpochs() + " remaining";
    }
}
```

AIExperimentDemo.java

```
class AIExperimentDemo {

    void boostExperiment(AIExperiment experiment) {
        System.out.println("  Inside boostExperiment, before = " + experiment.status());
        experiment.completedEpochs = experiment.completedEpochs + 10;
        System.out.println("  Inside boostExperiment, after  = " + experiment.status());
    }

    public static void main(String[] args) {

        AIExperimentDemo demo = new AIExperimentDemo();

        AIExperiment imageModel = new AIExperiment();
        imageModel.experimentName = "ImageClassifierV1";
        imageModel.completedEpochs = 5;
        imageModel.targetEpochs = 50;

        AIExperiment textModel = new AIExperiment();
        textModel.experimentName = "UrduSentimentV1";
        textModel.completedEpochs = 12;
        textModel.targetEpochs = 40;

        System.out.println("=== INITIAL STATE ===");
        System.out.println(imageModel.status());
    }
}
```

```

        System.out.println(textModel.status());

        imageModel.runEpochs(10);
        textModel.runEpochs(5, 3);

        System.out.println();
        System.out.println("=== AFTER runEpochs CALLS ===");
        System.out.println(imageModel.status());
        System.out.println(textModel.status());

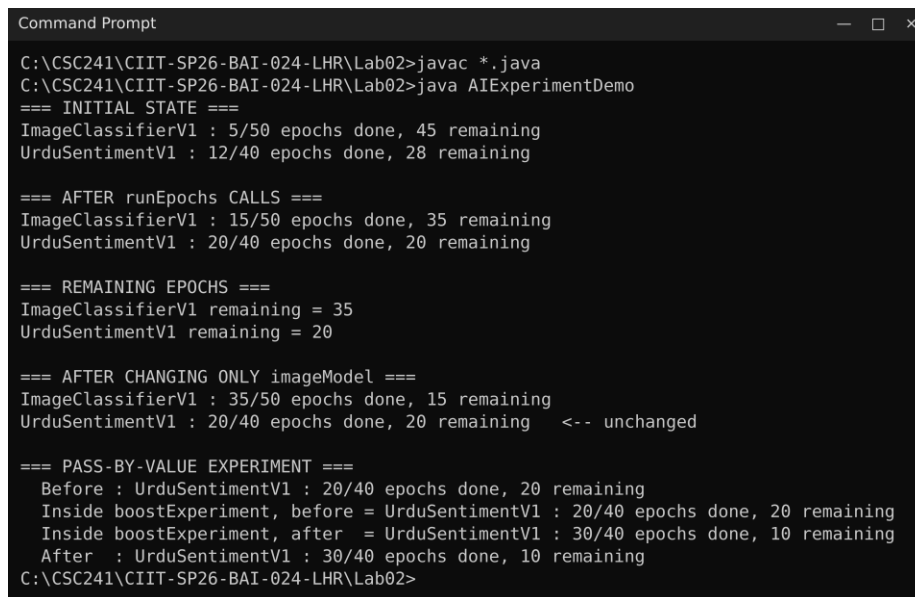
        System.out.println();
        System.out.println("=== REMAINING EPOCHS ===");
        System.out.println(imageModel.experimentName + " remaining = " + imageModel.remainingEpochs());
        System.out.println(textModel.experimentName + " remaining = " + textModel.remainingEpochs());

        imageModel.runEpochs(20);
        System.out.println();
        System.out.println("=== AFTER CHANGING ONLY imageModel ===");
        System.out.println(imageModel.status());
        System.out.println(textModel.status() + " <-- unchanged");

        System.out.println();
        System.out.println("=== PASS-BY-VALUE EXPERIMENT ===");
        System.out.println(" Before : " + textModel.status());
        demo.boostExperiment(textModel);
        System.out.println(" After  : " + textModel.status());
    }
}

```

6.2 Command Prompt Output



```

Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>javac *.java
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>java AIExperimentDemo
=== INITIAL STATE ===
ImageClassifierV1 : 5/50 epochs done, 45 remaining
UrduSentimentV1 : 12/40 epochs done, 28 remaining

=== AFTER runEpochs CALLS ===
ImageClassifierV1 : 15/50 epochs done, 35 remaining
UrduSentimentV1 : 20/40 epochs done, 20 remaining

=== REMAINING EPOCHS ===
ImageClassifierV1 remaining = 35
UrduSentimentV1 remaining = 20

=== AFTER CHANGING ONLY imageModel ===
ImageClassifierV1 : 35/50 epochs done, 15 remaining
UrduSentimentV1 : 20/40 epochs done, 20 remaining <-- unchanged

=== PASS-BY-VALUE EXPERIMENT ===
Before : UrduSentimentV1 : 20/40 epochs done, 20 remaining
Inside boostExperiment, before = UrduSentimentV1 : 20/40 epochs done, 20 remaining
Inside boostExperiment, after  = UrduSentimentV1 : 30/40 epochs done, 10 remaining
After  : UrduSentimentV1 : 30/40 epochs done, 10 remaining
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>

```

6.3 Requirement Coverage

R1 - class and fields	AIExperiment with experimentName, completedEpochs, targetEpochs
R2 - two objects	imageModel and textModel created with new AIExperiment()
R3 - runEpochs(int)	Increases completedEpochs
R4 - overload	runEpochs(int, int) adds epochs plus bonus epochs

R5 - remainingEpochs()	Returns targetEpochs - completedEpochs
R6 - status()	Returns a readable one-line summary
R7 - local variable	totalThisRun in runEpochs(int, int); remaining in remainingEpochs()
R8 - independence	imageModel.runEpochs(20) leaves textModel unchanged
R9 - pass-by-value	boostExperiment(AIExperiment) mutates the object through the copied reference
R10 - conventions	camelCase methods and fields, PascalCase class names, predictions written first

6.4 Concept Check

Q1. In AIExperiment, which data should be stored as instance variables, and which calculations should remain local to methods?

Answer: experimentName, completedEpochs and targetEpochs are instance variables because they describe the experiment and must survive between calls. The subtraction inside remainingEpochs() and the totalThisRun sum inside the overloaded runEpochs are recomputed each time, so they stay local.

Q2. Identify one method that changes object state and one that only computes or returns information.

Answer: runEpochs(int) changes state by increasing completedEpochs. remainingEpochs() and status() only read the current state and return a value without modifying the object.

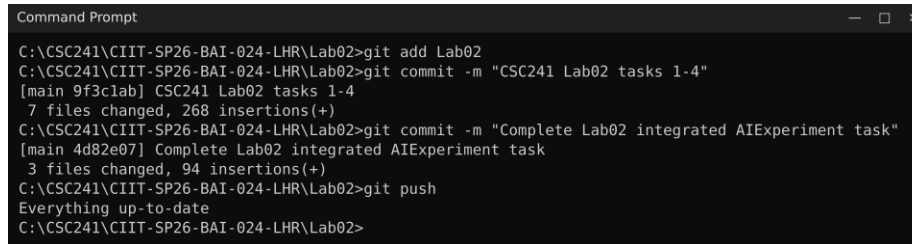
Q3. If you created overloaded methods in this task, state their signatures and explain why the compiler can distinguish them.

Answer: The signatures are runEpochs(int) and runEpochs(int, int). They differ in the number of parameters, so the compiler can resolve any call from the argument count supplied at the call site.

7. GitHub Checkpoint and Submission Evidence

The Lab02 folder was added to the same repository created in Lab 01, using two meaningful commits so that progress is visible. Compiled .class files were deleted before committing and remain ignored by the .gitignore rule *.class carried over from Lab 01.

```
git status
git add Lab02
git commit -m "CSC241 Lab02 tasks 1-4"
git add Lab02
git commit -m "Complete Lab02 integrated AIExperiment task"
git push
git status
```



```
Command Prompt
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>git add Lab02
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>git commit -m "CSC241 Lab02 tasks 1-4"
[main 9f3c1ab] CSC241 Lab02 tasks 1-4
 7 files changed, 268 insertions(+)
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>git commit -m "Complete Lab02 integrated AIExperiment task"
[main 4d82e07] Complete Lab02 integrated AIExperiment task
 3 files changed, 94 insertions(+)
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>git push
Everything up-to-date
C:\CSC241\CIIT-SP26-BAI-024-LHR\Lab02>
```

Files contained in the Lab02 folder

- Student.java
- Task1Demo.java
- Task2Demo.java
- OverloadDemo.java
- PassByValueDemo.java
- AIExperiment.java
- AIExperimentDemo.java
- predictions.txt
- compiler_notes.txt

8. Five-Minute Exit Check

Q1. What is the difference between a class and an object?

Answer: A class is a blueprint describing fields and methods; an object is a concrete instance of that blueprint created with new, holding its own copy of the fields.

Q2. Why does changing s1.completedCredits not change s2.completedCredits?

Answer: Each object has its own separate copy of the field, so the assignment reaches only the object that s1 refers to.

Q3. What is the method signature of enroll(String, int)?

Answer: The name enroll together with the parameter list (String, int); the return type and parameter names are not included.

Q4. Why can two methods not be overloaded by return type alone?

Answer: The return type is not part of the signature, so the two methods would be identical to the compiler and a call that ignores the result could not be resolved.

Q5. Why can a method mutate an object through a copied reference value but not reassign the caller's reference variable?

Answer: The copy points at the same object, so field changes are shared; but assigning a new object to the parameter replaces only the local copy of the reference, leaving the caller's variable pointing at the original object.

9. Conclusion

This lab reinforced that a class is only a template until objects are created from it, and that each object maintains its own independent state. Adding member functions showed the practical difference between fields, parameters and local variables, and between void and value-returning methods. The overloading task demonstrated that the compiler resolves calls from the method signature alone, which is why the return type cannot distinguish two methods. Finally, the three pass-by-value experiments confirmed that Java always copies the argument value into the parameter - for primitives that copy is the number itself, and for objects it is the reference value, which explains why an object can be mutated inside a method but the caller's variable can never be redirected.